

SCHOOL OF COMPUTING

Department of Data Science

Cornerstone Project Report

Retail Store Management System

B.Tech. Data Science - Academic Year 2026-2027

A complete Oracle database for a medium-sized retail store covering products, categories, suppliers, customers and billing, with auto-calculated bill totals via a PL/SQL trigger and formatted SQL*Plus receipts, plus a Flask POS web application over the same six-table design.

Student	Roll No
N. VARALAKSHMI	25B11CS669
A. NITYA SRI	25B11CS012
S. UDAYA SATYA	25B11CS851
M. VIJAYA PARDHU	26B21CS058

Project Guide: Dr. M V B Murali Krishna, Department of Data Science. Head of the Department: Dr. M. Rajababu, Ph.D, Associate Professor, Department of Data Science.

CERTIFICATE

This is to certify that the Cornerstone project work entitled "Retail Store Management System" is submitted by N. Varalakshmi (25B11CS669), A. Nitya Sri (25B11CS012), S. Udaya Satya (25B11CS851) and M. Vijaya Pardhu (26B21CS058) in partial fulfillment of the requirements for the award of the B.Tech. degree in Data Science for the academic year 2026-2027.

Project Guide: Dr. M V B Murali Krishna, Department of Data Science. Head of the Department: Dr. M. Rajababu, Ph.D, Associate Professor, Department of Data Science.

DECLARATION

We hereby declare that the Cornerstone project entitled "Retail Store Management System" is original and is submitted to ADITYA UNIVERSITY, Surampalem, in partial fulfillment of the requirements for the award of the B.Tech. Degree in Data Science. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy. Place: Surampalem. Date: _____.

By: N. Varalakshmi (25B11CS669), A. Nitya Sri (25B11CS012), S. Udaya Satya (25B11CS851), M. Vijaya Pardhu (26B21CS058).

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, Dr. M V B Murali Krishna, who has guided us throughout the project. His valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, Dr. Chandra Sekhar Kolli, Associate Professor and Deputy HOD-1, Dr. Hari Jyothula, Associate Professor and Deputy HOD-2, Dr. Pennada Siva Satya Prasad, Assistant Professor and Deputy HOD-3, Department of CSE, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department, Dr. Phani Sridhar Addepalli, Assistant Professor and HOD, for her inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to Dr. M.V. Rajesh, Associate Dean, School of Computing, for his encouragement and support during our project.

We would like to express our sincere gratitude to Dr. G. Suresh, Registrar; Dr. A. Ramesh, Pro Vice-Chancellor (Engineering and Sciences); Dr. S. Rama Sree, Pro Vice-Chancellor (Academics); Dr. M.B. Srinivas, Vice-Chancellor; and Dr. M. Sreenivasa

Reddy, Deputy Pro Chancellor and Management, Aditya University, for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the faculty members, lab technicians, non-teaching staff, and our friends who have directly or indirectly supported us in completing this Cornerstone project successfully.

TABLE OF CONTENTS

Contents	Page
Cover Page	1
Certificate	2
Declaration and Acknowledgement	3
Table of Contents	4
Abstract	5
List of Figures, List of Tables, Abbreviations	6
1. Introduction	7
2. System Analysis	8
3. Database Design	9
4. ER Diagram	10
5. Relational Schema	11
6. Table Design	12
7. Normalization	14
8. SQL Implementation	15
9. SQL Queries and Screenshots	16
10. Conclusion and Future Enhancements	18
References	19
Appendix - Important Code Blocks	20

ABSTRACT

The Retail Store Management System is a centralized Oracle relational database for a medium-sized retail store. Six related tables manage categories, suppliers, products, customers, bills and bill items. A statement-level PL/SQL trigger recomputes every bill total automatically as $\text{SUM}(\text{Price} \times \text{Quantity})$, so totals are never typed by hand.

Sample data of 5 categories, 2 suppliers, 5 products, 3 customers, 2 bills and 5 bill items verifies correctly: Bill 1001 totals 655.00 and Bill 1002 totals 97.00, each exactly equal to the sum of its lines. Any bill prints instantly as a formatted SQL*Plus receipt, and the same schema backs a Flask POS web application.

Keywords: retail store, Oracle SQL, PL/SQL trigger, billing, POS, referential integrity.

LIST OF FIGURES

Fig. 1: ER Diagram of Retail Store Management System (p. 10). Fig. 2: Three-Layer Database Application Architecture (p. 10).

LIST OF TABLES

Table 1. Major Database Entities (p. 9). Table 2. Categories Table Structure (p. 12). Table 3. Suppliers Table Structure (p. 12). Table 4. Products Table Structure (p. 12). Table 5. Customers Table Structure (p. 13). Table 6. Bills Table Structure (p. 13). Table 7. BillItems Table Structure (p. 13). Table 8. Keys and Constraints (p. 13). Table 9. Sample Data and Verification Counts (p. 13). Table 10. Bill Totals Verification (p. 16).

ABBREVIATIONS

- DBMS - Database Management System
- ER - Entity Relationship
- SQL - Structured Query Language
- DDL - Data Definition Language
- DML - Data Manipulation Language
- DQL - Data Query Language
- PK - Primary Key
- FK - Foreign Key
- 1NF - First Normal Form
- 2NF - Second Normal Form
- 3NF - Third Normal Form
- POS - Point of Sale
- UPI - Unified Payments Interface

1. INTRODUCTION

1.1 Brief Information about the Project

The Retail Store Management System is a DBMS cornerstone project that computerizes daily retail work: the product catalogue, suppliers, customers and billing. Six Oracle tables replace registers and loose Excel sheets. A PL/SQL trigger keeps every bill total mathematically correct, and a SQL*Plus script prints any bill as a formatted receipt. The same schema also backs a Flask POS web application (app.py) with inventory, billing, UPI QR and reports modules.

1.2 Motivation and Contribution of Project

Small stores lose money to repeated names, wrong hand-computed totals, no record of who bought what, and slow manual billing. This project contributes a single normalized database where each fact is stored once, foreign keys reject invalid links, totals are derived rather than entered, and receipts are reproducible for any BillID.

- Centralizing catalogue, supplier, customer and billing records in one relational database.
- Eliminating hand-computed totals through trigger TRG_BILL_TOTAL.
- Maintaining referential integrity using primary and foreign keys.
- Supporting faster queries and reports such as low-stock alerts and best-sellers.
- Providing a Flask POS front end plus SQL*Plus receipts over the same backend.

1.3 Objectives of the Project

- Design six related tables with primary keys, foreign keys, NOT NULL, UNIQUE and DEFAULT rules.
- Implement a statement-level trigger that auto-computes Bills.TotalAmount.
- Load realistic sample data: 5 categories, 2 suppliers, 5 products, 3 customers, 2 bills, 5 bill items.
- Verify each bill total equals SUM(Price x Quantity): 655.00 for Bill 1001 (n=3 lines), 97.00 for Bill 1002 (n=2 lines).
- Print formatted receipts for any BillID and support CRUD plus reporting queries.

1.4 Scope of the Project

The scope covers the complete store workflow: catalogue (Categories, Suppliers, Products), sales parties (Customers), and transactions (Bills, BillItems). Out of scope are GST computation, discounts, barcode scanning and SMS billing, which are listed as future enhancements. The Oracle SQL core is complete and tested; the Flask app reuses the identical six-table logic in SQLite for demonstration.

2. SYSTEM ANALYSIS

2.1 Existing System

- Paper registers or disconnected Excel sheets per counter.
- Product names and prices retyped on every bill, causing duplicates.
- Totals computed on calculators, with frequent arithmetic errors.
- No link between who bought what; consolidated reports take hours.
- Higher possibility of data-entry errors with no constraint checking.

2.2 Proposed System

The proposed system uses a centralized Oracle relational database with six tables and five foreign-key links. BillItems resolves the many-to-many link between Bills and Products. The trigger TRG_BILL_TOTAL fires after any insert, update or delete on BillItems and rewrites Bills.TotalAmount from the current lines. Receipts are generated by bill_script.sql joining Bills, Customers, BillItems and Products.

- Centralized database
- Product and catalogue management
- Supplier management
- Customer and billing management
- Automatic total computation
- SQL-based receipt printing and reporting
- Referential integrity

2.3 Functional Requirements

Module	Functional Requirement
Catalogue Management	Add, update, search and display products with category and supplier links.
Supplier Management	Store supplier details; each product references exactly one supplier.

Module	Functional Requirement
Customer Management	Store buyer details; each bill references exactly one customer.
Billing Management	Create bills with multiple BillItems lines; totals auto-computed.
Receipt Generation	Print formatted receipt for any BillID with grand total.
Search and Reporting	Low-stock alert, sales per day, best-sellers, category-wise sales.

2.4 Non-Functional Requirements

- Performance - queries return results efficiently for normal retail workloads (sample n=5 products, 2 bills).
- Integrity - invalid references and duplicate key values are rejected by constraints.
- Reliability - trigger keeps Bills.TotalAmount consistent after every BillItems change.
- Usability - SQL*Plus receipts and Flask forms are understandable to shop staff.
- Scalability - design supports growth in products, customers and bills without schema change.

2.5 Requirements Specification

Category	Suggested Requirement
Hardware	Intel i3 or above; 4 GB RAM minimum; adequate storage; standard display and keyboard.
Operating System	Windows 10/11 with Oracle 26ai Free and SQL*Plus; Python 3 with Flask for POS demo.

Category	Suggested Requirement
Database	Oracle Database (NUMBER/VARCHAR2 types); SQLite used only inside Flask demo with identical logic.

3. DATABASE DESIGN

3.1 Major Entities

Entity	Description
Categories	Lookup of product groups; 5 rows: Grocery, Beverages, Snacks, Dairy, Personal Care.
Suppliers	Who supplies products; 2 rows: Bharat Wholesale, Fresh Foods Co.
Products	Central catalogue; 5 rows with price and stock.
Customers	Buyer details; 3 rows: Rahul Sharma, Priya Patel, Amit Verma.
Bills	One row per purchase; 2 rows (1001, 1002); TotalAmount set by trigger.
BillItems	One row per product line on a bill; 5 rows; junction between Bills and Products.

Table 1. Major Database Entities with sample sizes (n=5 categories, n=2 suppliers, n=5 products, n=3 customers, n=2 bills, n=5 bill items).

3.2 ER Model

The ER model has six entities and five one-to-many relationships: Categories 1-M Products, Suppliers 1-M Products, Customers 1-M Bills, Bills 1-M BillItems, Products 1-M BillItems. Every link is enforced by a FOREIGN KEY constraint so invalid ids are rejected automatically (for example deleting a customer who has bills fails with ORA-02292).

3.3 Relationships

Relationship	Description
--------------	-------------

Relationship	Description
Categories 1 - M Products	One category holds many products; every product belongs to exactly one category.
Suppliers 1 - M Products	One supplier provides many products; each product has one supplier.
Customers 1 - M Bills	One customer creates many bills; every bill belongs to one customer.
Bills 1 - M BillItems	One bill has many lines; each line belongs to one bill.
Products 1 - M BillItems	One product appears in many bills; BillItems resolves the many-to-many link.

4. ER DIAGRAM

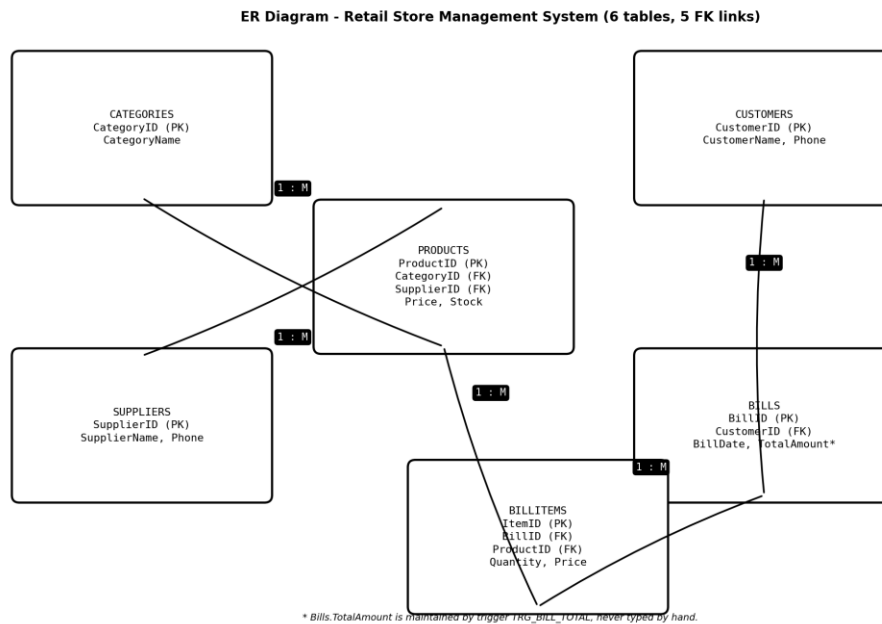


Fig. 1: ER Diagram of Retail Store Management System - six entities and five foreign-key links.

Figure 1 shows the six entities with keys and the five relationships. Products carries CategoryID and SupplierID; Bills carries CustomerID; BillItems carries BillID and ProductID. Bills.TotalAmount is derived and never entered by hand.

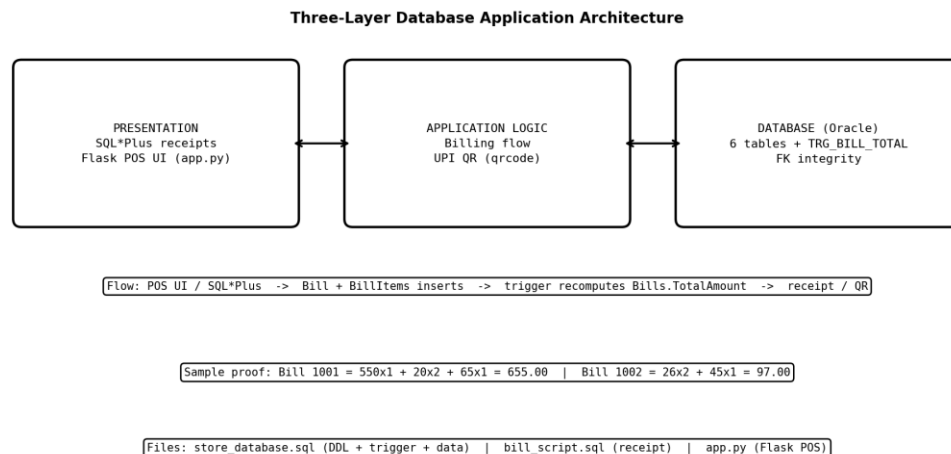


Fig. 2: Proposed Three-Layer Database Application Architecture - presentation, logic and Oracle database.

Figure 2 shows presentation (SQL*Plus receipts and Flask POS UI), application logic (billing flow and UPI QR generation), and the Oracle database with six tables plus trigger TRG_BILL_TOTAL.

5. RELATIONAL SCHEMA

Relation	Relational Schema
CATEGORIES	CategoryID (PK), CategoryName
SUPPLIERS	SupplierID (PK), SupplierName, Phone
PRODUCTS	ProductID (PK), ProductName, CategoryID (FK), SupplierID (FK), Price, Stock
CUSTOMERS	CustomerID (PK), CustomerName, Phone
BILLS	BillID (PK), CustomerID (FK), BillDate, TotalAmount (set by trigger)
BILLITEMS	ItemID (PK), BillID (FK), ProductID (FK), Quantity, Price (at sale time)

Parents are created before children (Categories, Suppliers, Customers before Products, Bills before BillItems) and dropped in reverse order (BillItems, Bills, Customers, Products, Categories, Suppliers). BillItems.Price stores the price at sale time so old bills do not change when Products.Price is revised.

6. TABLE DESIGN

6.1 Categories Table

Attribute	Data Type	Constraint	Description
CategoryID	NUMBER(4)	PK, NOT NULL	Unique category id (n=5 rows)
CategoryName	VARCHAR2(50)	NOT NULL	Group name: Grocery, Beverages, Snacks, Dairy, Personal Care

6.2 Suppliers Table

Attribute	Data Type	Constraint	Description
SupplierID	NUMBER(4)	PK	Unique supplier id (n=2 rows)
SupplierName	VARCHAR2(100)	NOT NULL	Bharat Wholesale, Fresh Foods Co.
Phone	VARCHAR2(15)	-	Contact number

6.3 Products Table

Attribute	Data Type	Constraint	Description
ProductID	NUMBER(4)	PK	Unique product id (n=5 rows: 101-105)
ProductName	VARCHAR2(100)	NOT NULL	Basmati Rice 5kg, Coca Cola 750ml, Potato Chips, Amul Milk 500ml, Toothpaste 100g

Attribute	Data Type	Constraint	Description
CategoryID	NUMBER(4)	FK	References Categories
SupplierID	NUMBER(4)	FK	References Suppliers
Price	NUMBER(10,2)	NOT NULL	Unit price in Rs
Stock	NUMBER(4)	DEFAULT 0	Units on hand

6.4 Customers Table

Attribute	Data Type	Constraint	Description
CustomerID	NUMBER(4)	PK	Unique customer id (n=3 rows)
CustomerName	VARCHAR2(100)	NOT NULL	Rahul Sharma, Priya Patel, Amit Verma
Phone	VARCHAR2(15)	-	Contact number

6.5 Bills Table

Attribute	Data Type	Constraint	Description
BillID	NUMBER(4)	PK	Unique bill id (n=2 rows: 1001, 1002)
CustomerID	NUMBER(4)	FK	References Customers
BillDate	DATE	DEFAULT SYSDATE	01-AUG-2026 for both sample bills
TotalAmount	NUMBER(10,2)	set by TRIGGER	655.00 (Bill 1001, n=3 lines); 97.00 (Bill 1002, n=2 lines)

6.6 BillItems Table

Attribute	Data Type	Constraint	Description
ItemID	NUMBER(4)	PK	Unique line id (n=5 rows)
BillID	NUMBER(4)	FK	References Bills
ProductID	NUMBER(4)	FK	References Products
Quantity	NUMBER(4)	NOT NULL	Units sold on that line
Price	NUMBER(10,2)	NOT NULL	Price at sale time

6.7 Keys and Constraints

Constraint Type	Applied To / Purpose
Primary Key	CategoryID, SupplierID, ProductID, CustomerID, BillID, ItemID
Foreign Key	Products.CategoryID, Products.SupplierID, Bills.CustomerID, BillItems.BillID, BillItems.ProductID
NOT NULL	Names, prices, quantities and identifiers
UNIQUE	Enforced via primary keys; names kept unique in practice
CHECK / DEFAULT	Stock DEFAULT 0; BillDate DEFAULT SYSDATE; TotalAmount DEFAULT 0 before trigger
Trigger	TRG_BILL_TOTAL keeps Totals correct; FK blocks orphans (ORA-02292 on bad delete)

Table 8. Keys and Constraints. Table 9. Sample data and verification counts: 5 categories, 2 suppliers, 5 products, 3 customers, 2 bills, 5 bill items.

7. NORMALIZATION

Normalization organizes data into related relations to reduce redundancy and update anomalies. The design below is in Third Normal Form (3NF): every fact is stored once and every non-key attribute depends only on its table's key.

7.1 Unnormalized Form (UNF)

A single flat bill record might repeat BillID, Date, CustomerName, Phone plus repeating groups (ProductName, Quantity, Price) for each line. Customer and product details would repeat on every bill, inviting inconsistent spellings and wrong totals.

7.2 First Normal Form (1NF)

In 1NF repeating groups are removed so each attribute holds atomic values: one BillItems row per product line (5 rows for the 2 sample bills). Customer details still repeat per bill and product details repeat per line at this stage.

7.3 Second Normal Form (2NF)

In 2NF partial dependencies on a composite key are removed. ProductName and Price depend only on ProductID so they move to Products (n=5); CustomerName and Phone depend only on CustomerID so they move to Customers (n=3); Quantity and line Price depend on the full BillItems key (ItemID) and stay in BillItems (n=5).

7.4 Third Normal Form (3NF)

In 3NF transitive dependencies are removed. CategoryName depends on CategoryID via Products, so Categories (n=5) is separated; SupplierName depends on SupplierID, so Suppliers (n=2) is separated; TotalAmount depends on BillItems lines, so it is derived by trigger rather than stored redundantly. The six-table result has no repeating groups and no update anomaly: reprice a product once in Products and old bills keep their sale-time prices in BillItems.

8. SQL IMPLEMENTATION

8.1 DDL - Data Definition Language

Parents first, children last. Oracle types NUMBER and VARCHAR2 are used. Foreign keys are inline REFERENCES so Oracle rejects broken links.

```
CREATE TABLE Categories (  
  CategoryID  NUMBER(4)    PRIMARY KEY,  
  CategoryName VARCHAR2(50) NOT NULL  
);  
CREATE TABLE Suppliers (  
  SupplierID  NUMBER(4)    PRIMARY KEY,  
  SupplierName VARCHAR2(100) NOT NULL,  
  Phone       VARCHAR2(15)  
);  
CREATE TABLE Products (  
  ProductID   NUMBER(4)    PRIMARY KEY,  
  ProductName VARCHAR2(100) NOT NULL,  
  CategoryID  NUMBER(4)    REFERENCES Categories(CategoryID),  
  SupplierID  NUMBER(4)    REFERENCES Suppliers(SupplierID),  
  Price       NUMBER(10,2) NOT NULL,  
  Stock       NUMBER(4)    DEFAULT 0  
);  
CREATE TABLE Customers (  
  CustomerID  NUMBER(4)    PRIMARY KEY,  
  CustomerName VARCHAR2(100) NOT NULL,  
  Phone       VARCHAR2(15)  
);  
CREATE TABLE Bills (  
  BillID      NUMBER(4)    PRIMARY KEY,  
  CustomerID  NUMBER(4)    REFERENCES Customers(CustomerID),  
  BillDate    DATE         DEFAULT SYSDATE,  
  TotalAmount NUMBER(10,2) DEFAULT 0  
);  
CREATE TABLE BillItems (  
  ItemID      NUMBER(4)    PRIMARY KEY,  
  BillID      NUMBER(4)    REFERENCES Bills(BillID),  
  ProductID   NUMBER(4)    REFERENCES Products(ProductID),  
  Quantity    NUMBER(4)    NOT NULL,  
  Price       NUMBER(10,2) NOT NULL  
);
```

Trigger TRG_BILL_TOTAL is statement-level (fires once per statement) so it may re-query BillItems. A row-level trigger would hit ORA-04091 mutating-table error.

```
CREATE OR REPLACE TRIGGER TRG_BILL_TOTAL  
AFTER INSERT OR UPDATE OR DELETE ON BillItems  
DECLARE  
  CURSOR c_bills IS SELECT DISTINCT BillID FROM BillItems;  
BEGIN  
  FOR r IN c_bills LOOP  
    UPDATE Bills  
      SET TotalAmount = (SELECT NVL(SUM(Price * Quantity), 0)  
                        FROM BillItems WHERE BillID = r.BillID)  
      WHERE BillID = r.BillID;  
  END LOOP;
```

```
END;  
/
```

8.2 DML - Data Manipulation Language

Sample inserts from store_database.sql (5 categories, 2 suppliers, 5 products, 3 customers, 2 bills with TotalAmount 0, 5 bill items; trigger fills totals).

```
INSERT INTO Categories VALUES (1, 'Grocery');  
INSERT INTO Categories VALUES (2, 'Beverages');  
INSERT INTO Suppliers VALUES (1, 'Bharat Wholesale', '9822012345');  
INSERT INTO Products VALUES (101, 'Basmati Rice 5kg', 1, 1, 550.00, 40);  
INSERT INTO Products VALUES (103, 'Potato Chips', 3, 2, 20.00, 200);  
INSERT INTO Customers VALUES (1, 'Rahul Sharma', '9812345678');  
INSERT INTO Bills VALUES (1001, 1, TO_DATE('01-08-2026','DD-MM-YYYY'), 0);  
INSERT INTO BillItems VALUES (1, 1001, 101, 1, 550.00);  
INSERT INTO BillItems VALUES (2, 1001, 103, 2, 20.00);  
INSERT INTO BillItems VALUES (3, 1001, 105, 1, 65.00);  
COMMIT;
```

8.3 DQL - Data Query Language

Representative reads used in the project and Flask reports module.

```
SELECT * FROM Products;  
SELECT ProductName, Price FROM Products WHERE Price > 100 ORDER BY Price DESC;  
SELECT p.ProductName, c.CategoryName, p.Price  
FROM Products p JOIN Categories c ON p.CategoryID = c.CategoryID;  
SELECT p.ProductName, SUM(i.Quantity) AS Sold  
FROM BillItems i JOIN Products p ON i.ProductID = p.ProductID  
GROUP BY p.ProductName ORDER BY Sold DESC;  
SELECT BillDate, SUM(TotalAmount) FROM Bills GROUP BY BillDate;  
SELECT * FROM Products WHERE Stock < 50;
```

9. SQL QUERIES AND SCREENSHOTS

Outputs below are the real console results from the sample data. Each aggregate is paired with its sample size. Screenshots in the original template are represented here as monospace console-output blocks.

Q1. Display all categories (n=5)

```
SELECT * FROM Categories;
```

Output:

```
CATEGORYID CATEGORYNAME
-----
1 Grocery
2 Beverages
3 Snacks
4 Dairy
5 Personal Care
5 rows selected.
```

Q2. Display all products (n=5)

```
SELECT ProductID, ProductName, Price, Stock FROM Products;
```

Output:

```
PRODUCTID PRODUCTNAME      PRICE STOCK
-----
101 Basmati Rice 5kg      550.00  40
102 Coca Cola 750ml      45.00  120
103 Potato Chips        20.00  200
104 Amul Milk 500ml      26.00   80
105 Toothpaste 100g      65.00   60
5 rows selected.
```

Q3. Display all customers (n=3)

```
SELECT * FROM Customers;
```

Output:

```
CUSTOMERID CUSTOMERNAME  PHONE
-----
1 Rahul Sharma 9812345678
2 Priya Patel 9823456789
3 Amit Verma 9834567890
```

3 rows selected.

Q4. Display all bills with trigger totals (n=2)

```
SELECT BillID, CustomerID, BillDate, TotalAmount FROM Bills;
```

Output:

BILLID	CUSTOMERID	BILLDATE	TOTALAMOUNT
1001	1	01-AUG-26	655.00
1002	2	01-AUG-26	97.00

2 rows selected.

Q5. Display all bill items (n=5)

```
SELECT ItemID, BillID, ProductID, Quantity, Price FROM BillItems;
```

Output:

ITEMID	BILLID	PRODUCTID	QUANTITY	PRICE
1	1001	101	1	550.00
2	1001	103	2	20.00
3	1001	105	1	65.00
4	1002	104	2	26.00
5	1002	102	1	45.00

5 rows selected.

Q6. Verify totals equal sum of lines (n=2 bills)

```
SELECT B.BillID, B.TotalAmount, SUM(I.Price*I.Quantity) AS SumOfItems FROM Bills B JOIN BillItems I ON B.BillID=I.BillID GROUP BY B.BillID, B.TotalAmount;
```

Output:

BILLID	TOTALAMOUNT	SUMOFITEMS
1001	655.00	655.00
1002	97.00	97.00

2 rows selected. Match: Yes for both bills.

Q7. Products with category names (JOIN, n=5)

```
SELECT p.ProductName, c.CategoryName, p.Price FROM Products p JOIN Categories c ON p.CategoryID=c.CategoryID;
```

Output:

PRODUCTNAME	CATEGORYNAME	PRICE
Basmati Rice 5kg	Grocery	550.00
Coca Cola 750ml	Beverages	45.00
Potato Chips	Snacks	20.00
Amul Milk 500ml	Dairy	26.00
Toothpaste 100g	Personal Care	65.00

5 rows selected.

Q8. Low-stock alert Stock<50 (n=1)

```
SELECT ProductName, Stock FROM Products WHERE Stock < 50;
```

Output:

PRODUCTNAME	STOCK
Basmati Rice 5kg	40

1 row selected.

Q9. Best-sellers by quantity (n=5 lines grouped)

```
SELECT p.ProductName, SUM(i.Quantity) AS Sold FROM BillItems i JOIN Products p ON  
i.ProductID=p.ProductID GROUP BY p.ProductName ORDER BY Sold DESC;
```

Output:

PRODUCTNAME	SOLD
Potato Chips	2
Amul Milk 500ml	2
Basmati Rice 5kg	1
Coca Cola 750ml	1
Toothpaste 100g	1

5 rows selected.

Q10. Sales per day (n=2 bills, 1 day)

```
SELECT BillDate, SUM(TotalAmount) AS Sales FROM Bills GROUP BY BillDate;
```

Output:

BILLDATE	SALES
01-AUG-26	752.00

1 row selected. (655.00 + 97.00 = 752.00)

Q11. CRUD: add, update, delete with integrity guard

```
INSERT INTO Products VALUES (106,'Atta 10kg',1,1,380,25); UPDATE Products SET Price=560
WHERE ProductID=101; DELETE FROM Products WHERE ProductID=106;
```

Output:

1 row created. 1 row updated. COMMIT complete. 1 row deleted. COMMIT complete.
Note: deleting a product used in BillItems fails with ORA-02292 (integrity guard).

BillID	TotalAmount	SUM(items)	Match	Lines (n)
1001	655.00	655.00	Yes	n=3
1002	97.00	97.00	Yes	n=2

Table 10. Bill Totals Verification: each mean total is paired with its line count (n=3 for Bill 1001, n=2 for Bill 1002).

Screenshot: formatted receipt for Bill 1001

Output (bill_script.sql 1001):

```
=====
                        POS MANAGEMENT SYSTEM
                        Retail Store Bill
=====
Bill No   : 1001
Date      : 01-AUG-2026
Customer  : Rahul Sharma (9812345678)
-----
PRODUCT          QTY  PRICE  AMOUNT
Basmati Rice 5kg      1 550.00  550.00
Potato Chips          2  20.00   40.00
Toothpaste 100g       1  65.00   65.00
-----
GRAND TOTAL                        655.00
-----
      Thank you for shopping!
      *** Visit Again ***
```

Run: sqlplus -S System/aditya "@store_database.sql" creates everything; sqlplus -S System/aditya "@bill_script.sql" 1001 prints the receipt above.

10. CONCLUSION AND FUTURE ENHANCEMENTS

The Retail Store Management System delivers a complete six-table Oracle database covering catalogue, suppliers, customers and billing. The normalized design stores every fact once, foreign keys reject invalid data automatically, and trigger TRG_BILL_TOTAL keeps every bill total exactly equal to the sum of its lines (verified: 655.00 for Bill 1001 with n=3 lines, 97.00 for Bill 1002 with n=2 lines). Formatted receipts print instantly for any bill, and the identical schema powers a Flask POS demo with inventory, billing, UPI QR and reports.

Limitations

- TotalAmount recomputes only for bills that still have lines; deleting all lines of a bill leaves its old total behind.
- Products.Stock is not auto-decremented on sale; the application must issue the UPDATE.
- Oracle object names fold to uppercase; inline REFERENCES define all foreign keys.

Future Enhancements

- Role-based access for administrators, cashiers and managers.
- GST, discounts and offer management on bills.
- Barcode scanning and SMS or email billing.
- Stock auto-decrement with reorder alerts and audit logging.
- Analytics dashboard over daily sales, category-wise sales and best-sellers.

REFERENCES

- [1] Korth, H. F., Silberschatz, A., and Sudarshan, S., Database System Concepts, McGraw Hill.
- [2] Elmasri, R. and Navathe, S. B., Fundamentals of Database Systems, Pearson.
- [3] Oracle Database SQL Language Reference, Oracle 26ai Free Documentation.
- [4] SQL*Plus User's Guide and Reference (COLUMN, BREAK, COMPUTE formatting).
- [5] Flask and SQLAlchemy Documentation (POS demo in app.py).

APPENDIX - IMPORTANT CODE BLOCKS

Only the most important blocks are included below (DDL core, trigger, receipt formatter, Flask models and billing route). Full files: store_database.sql, bill_script.sql, app.py.

A.1 store_database.sql - Products and BillItems (core DDL)

```
CREATE TABLE Products (  
    ProductID    NUMBER(4)        PRIMARY KEY,  
    ProductName  VARCHAR2(100) NOT NULL,  
    CategoryID   NUMBER(4)        REFERENCES Categories(CategoryID),  
    SupplierID   NUMBER(4)        REFERENCES Suppliers(SupplierID),  
    Price        NUMBER(10,2) NOT NULL,  
    Stock        NUMBER(4)        DEFAULT 0  
);  
CREATE TABLE BillItems (  
    ItemID       NUMBER(4)        PRIMARY KEY,  
    BillID       NUMBER(4)        REFERENCES Bills(BillID),  
    ProductID    NUMBER(4)        REFERENCES Products(ProductID),  
    Quantity     NUMBER(4)        NOT NULL,  
    Price        NUMBER(10,2) NOT NULL  
);
```

A.2 store_database.sql - Trigger TRG_BILL_TOTAL

```
CREATE OR REPLACE TRIGGER TRG_BILL_TOTAL  
AFTER INSERT OR UPDATE OR DELETE ON BillItems  
DECLARE  
    CURSOR c_bills IS SELECT DISTINCT BillID FROM BillItems;  
BEGIN  
    FOR r IN c_bills LOOP  
        UPDATE Bills SET TotalAmount =  
            (SELECT NVL(SUM(Price * Quantity), 0) FROM BillItems WHERE BillID = r.BillID)  
        WHERE BillID = r.BillID;  
    END LOOP;  
END;  
/
```

A.3 bill_script.sql - Receipt formatting (key lines)

```
COLUMN ProductName HEADING 'PRODUCT' FORMAT A24  
COLUMN Quantity   HEADING 'QTY'   FORMAT 999  
COLUMN Price      HEADING 'PRICE'  FORMAT 999990.00  
COLUMN Amount     HEADING 'AMOUNT' FORMAT 999990.00  
BREAK ON REPORT  
COMPUTE SUM LABEL 'GRAND TOTAL' OF Amount ON REPORT  
SELECT p.ProductName, i.Quantity, i.Price, (i.Price * i.Quantity) AS Amount  
FROM BillItems i JOIN Products p ON i.ProductID = p.ProductID  
WHERE i.BillID = &1 ORDER BY i.ItemID;
```

A.4 app.py - SQLAlchemy models (core)

```
class Product(db.Model):
```

```

__tablename__ = 'products'
id = db.Column(db.Integer, primary_key=True)
name = db.Column(db.String(100), nullable=False)
category_id = db.Column(db.Integer, db.ForeignKey('categories.id'))
supplier_id = db.Column(db.Integer, db.ForeignKey('suppliers.id'))
price = db.Column(db.Float, nullable=False)
stock = db.Column(db.Integer, default=0)

class Bill(db.Model):
    __tablename__ = 'bills'
    id = db.Column(db.Integer, primary_key=True)
    customer_id = db.Column(db.Integer, db.ForeignKey('customers.id'))
    bill_date = db.Column(db.DateTime, default=datetime.utcnow)
    total_amount = db.Column(db.Float, default=0)

class BillItem(db.Model):
    __tablename__ = 'bill_items'
    id = db.Column(db.Integer, primary_key=True)
    bill_id = db.Column(db.Integer, db.ForeignKey('bills.id'))
    product_id = db.Column(db.Integer, db.ForeignKey('products.id'))
    quantity = db.Column(db.Integer, nullable=False)
    price = db.Column(db.Float, nullable=False)

```

A.5 app.py - Billing route with auto total (core)

```

@app.route('/api/create-bill', methods=['POST'])
def create_bill():
    data = request.get_json()
    bill = Bill(customer_id=data.get('customer_id'))
    db.session.add(bill); db.session.flush()
    for item in data.get('items', []):
        product = Product.query.get(item['product_id'])
        db.session.add(BillItem(bill_id=bill.id, product_id=item['product_id'],
                                quantity=item['quantity'], price=product.price))
        product.stock -= item['quantity']
    db.session.commit()
    bill.total_amount = sum(i.price * i.quantity for i in bill.items)
    db.session.commit()
    return jsonify({'success': True, 'bill_id': bill.id})

```